Playwright Stabilization Assessment Report

Submitted By

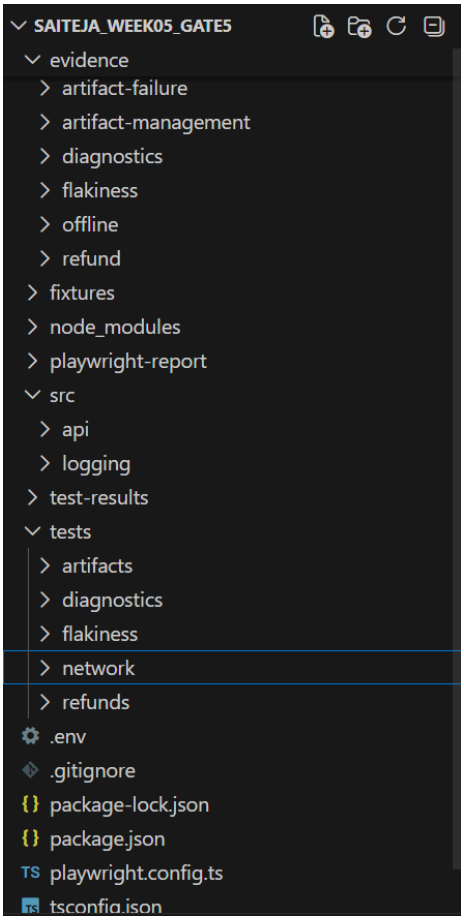
Name: Saiteja Kodi

Technology: Playwright with TypeScript

Repository:

Note:
Meaningful comments have been added throughout the test files to improve readability and make the test flow easier to understand. This report presents each assessment scenario using the **Diagnose → Fix → Evidence → Explain** approach, supported by relevant screenshots.

Code Structure:



1. Objective

The objective of this assessment is to stabilize the Playwright automation suite by identifying failures, diagnosing their root causes, validating fixes, and providing supporting evidence. The assessment focuses on writing reliable tests and demonstrating debugging maturity.

---

2. Assessment Overview

This assessment covers the following topics:

- Flakiness Analysis
- Diagnostic Logging
- Artifact Management
- Offline & Resilient UI
- Returns & Refund Testing

The overall approach followed throughout the assessment is:

Diagnose → Fix → Evidence → Explain

---

3. Tools Used

- Playwright
- TypeScript
- Node.js
- Visual Studio Code
- Git & GitHub

---

4. Flakiness Analysis

Command to run in terminal:

npx playwright test makeItFlaky.spec.ts –repeat-each=50

Test File: makeItFlaky.spec.ts

Scenario 1 – Race Condition

Issue

The cart total was verified before the asynchronous request completed.

Diagnosis

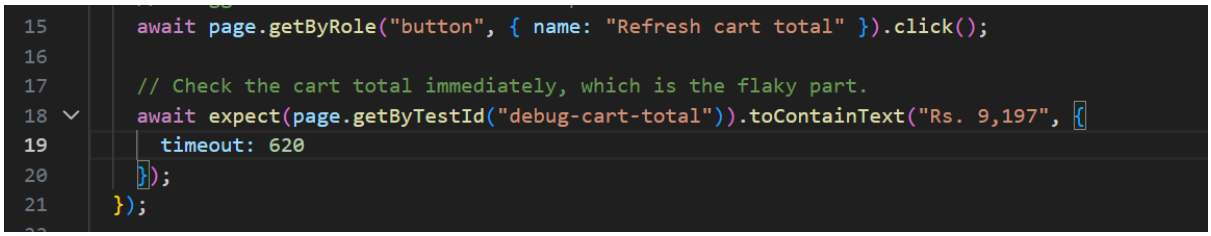
The assertion executed before the UI finished updating, causing intermittent failures.

Fix

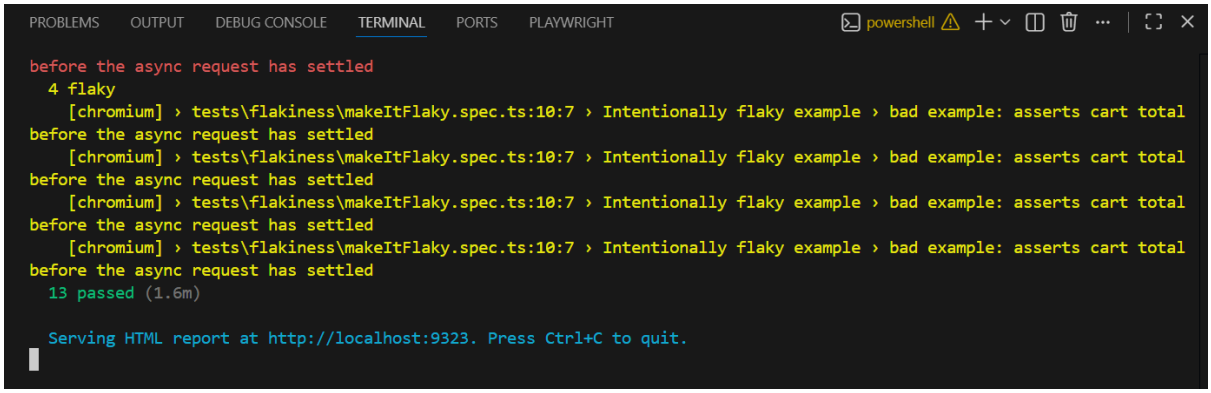
Use Playwright's web-first assertions and wait for the UI state instead of using very short timeouts.

Evidence

- Screenshot of failed test (Before)

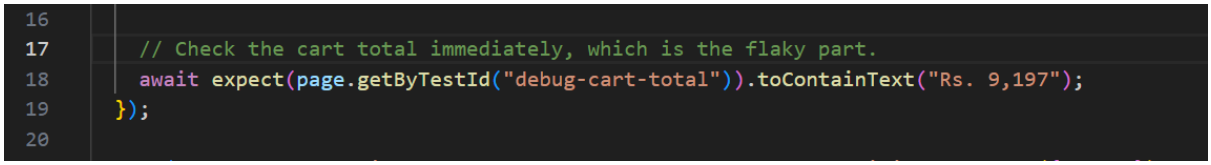


```
15    await page.getByRole("button", { name: "Refresh cart total" }).click();
16
17    // Check the cart total immediately, which is the flaky part.
18    await expect(page.getByTestId("debug-cart-total")).toContainText("Rs. 9,197", {
19      timeout: 620
20    });
21  });
```

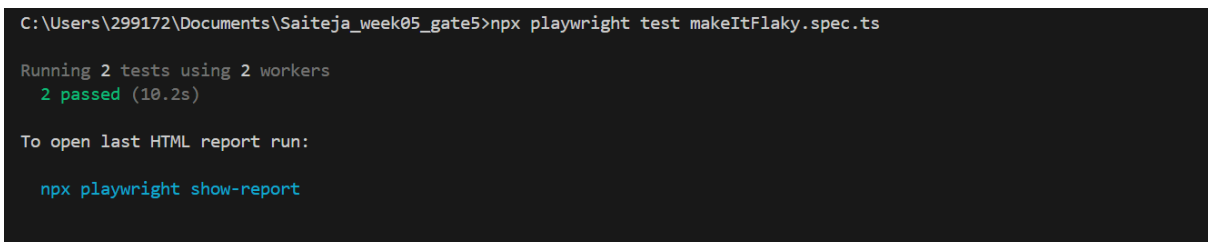


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PLAYWRIGHT        powershell

before the async request has settled
  4 flaky
    [chromium] › tests\flakiness\makeItFlaky.spec.ts:10:7 › Intentionally flaky example › bad example: asserts cart total
before the async request has settled
    [chromium] › tests\flakiness\makeItFlaky.spec.ts:10:7 › Intentionally flaky example › bad example: asserts cart total
before the async request has settled
    [chromium] › tests\flakiness\makeItFlaky.spec.ts:10:7 › Intentionally flaky example › bad example: asserts cart total
before the async request has settled
    [chromium] › tests\flakiness\makeItFlaky.spec.ts:10:7 › Intentionally flaky example › bad example: asserts cart total
before the async request has settled
  13 passed (1.6m)

  Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

- Screenshot of stable execution (After)

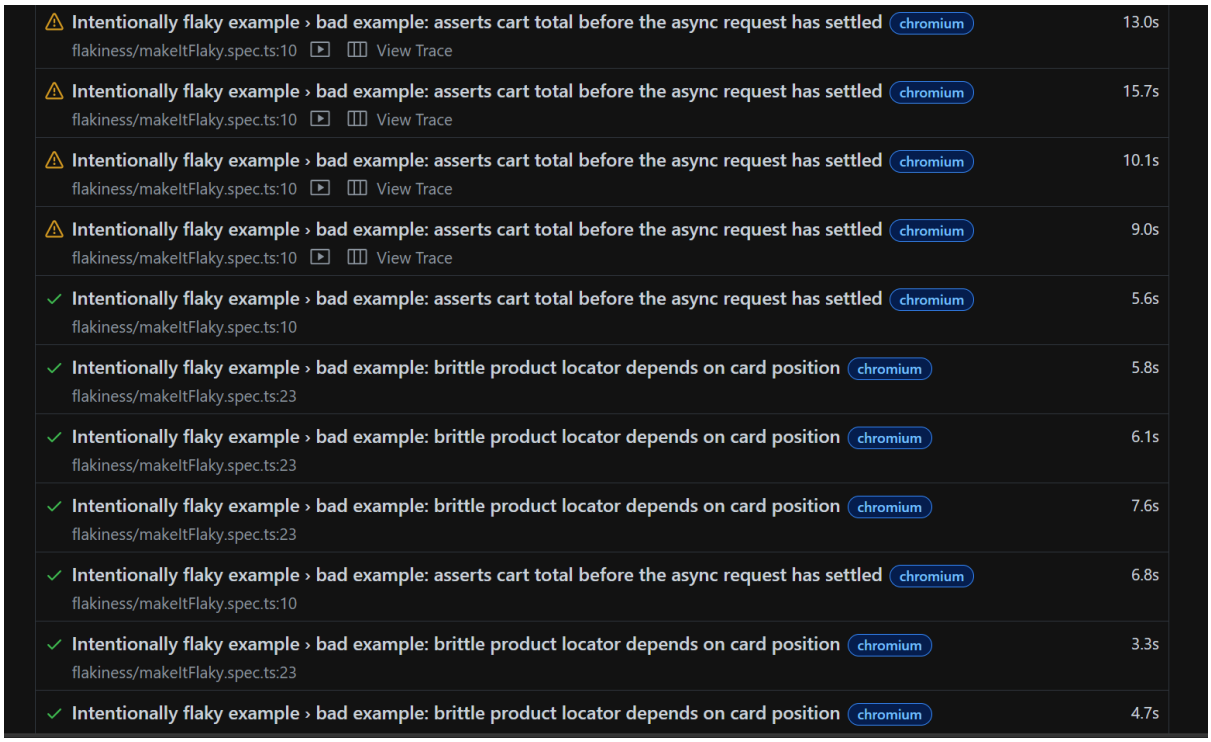


```
16
17    // Check the cart total immediately, which is the flaky part.
18    await expect(page.getByTestId("debug-cart-total")).toContainText("Rs. 9,197");
19  });
20
```

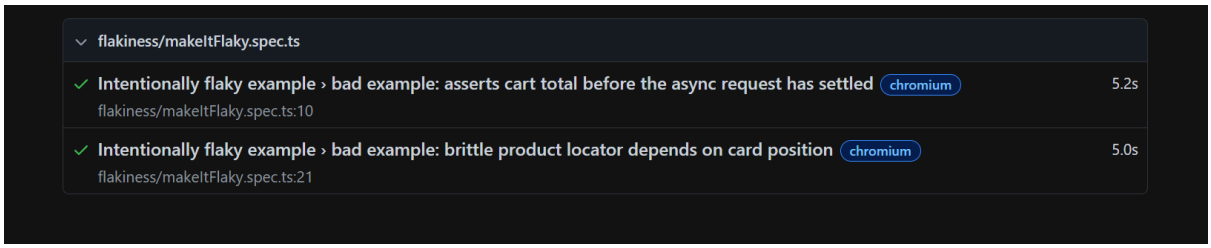


```
C:\Users\299172\Documents\Saiteja_week05_gate5>npx playwright test makeItFlaky.spec.ts

Running 2 tests using 2 workers
  2 passed (10.2s)

To open last HTML report run:

  npx playwright show-report
```

- HTML Report screenshot (Before)



- HTML Report screenshot (After)



Scenario 2 – Brittle Locator

Issue

The locator depended on the position of a product card.

Diagnosis

Any UI order change caused the test to fail.

Fix

Use stable locators such as getByRole() or getByTestId().

Evidence

- Screenshot of failure (Before)

```
24
25    // Navigate to a product using a fragile position-based selector.
26    await page.locator(".product-grid article:nth-child(2) a").click();
27
28    // Assert that the expected product page is visible.
29    await expect(
30      page.getByRole("heading", { name: "Travel Backpack" })
31    ).toBeVisible();
```

- Screenshot after using stable locator (After)

```
24
25    // Navigate to a product using a fragile position-based selector.
26    await page.getByRole("link", {name: "View Travel Backpack"}).click();
27
28    // Assert that the expected product page is visible.
29    await expect(
30      page.getByRole("heading", { name: "Travel Backpack" })
31    ).toBeVisible();
32  });
```

---

5. Diagnostic Logging

Command to run in terminal:

npx playwright test diagnostics-logging.spec.ts

Test File: diagnostics-logging.spec.ts

Purpose

To improve debugging by capturing structured logs for every important step in the checkout flow.

Implemented

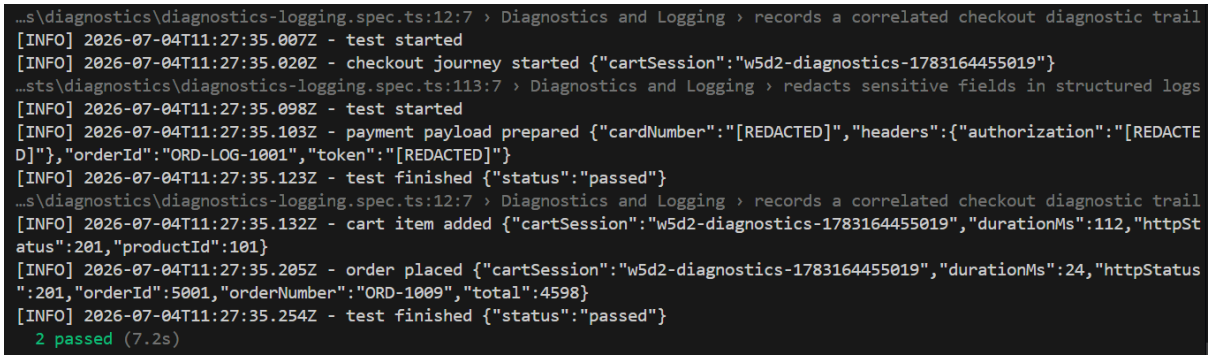
- Correlation ID
- Structured logging
- Response time logging
- Sensitive data redaction

Benefits

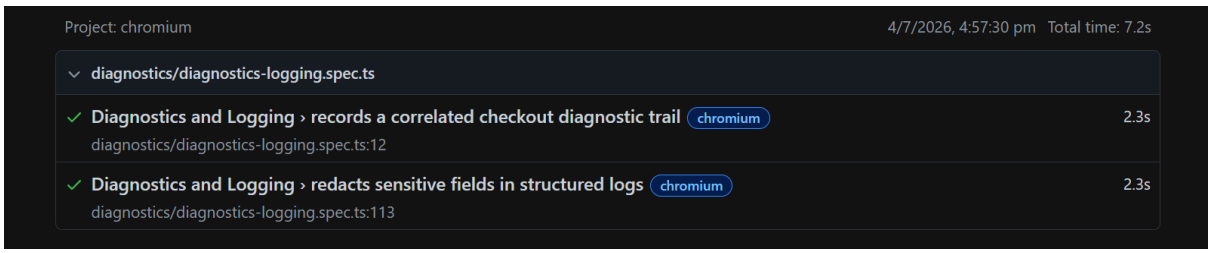
- Easy request tracing
- Faster debugging
- Secure logging without exposing sensitive information

Evidence

- Correlation ID and Redacted log screenshot of the terminal



- HTML Report



---

6. Artifact Management

Command to run in terminal:

npx playwright test artifact-management.spec.ts

npx playwright test artifact-failure.spec.ts

Test Files

- artifact-management.spec.ts
- artifact-failure.spec.ts

Purpose

To capture useful debugging evidence whenever tests pass or fail.

Artifacts Captured

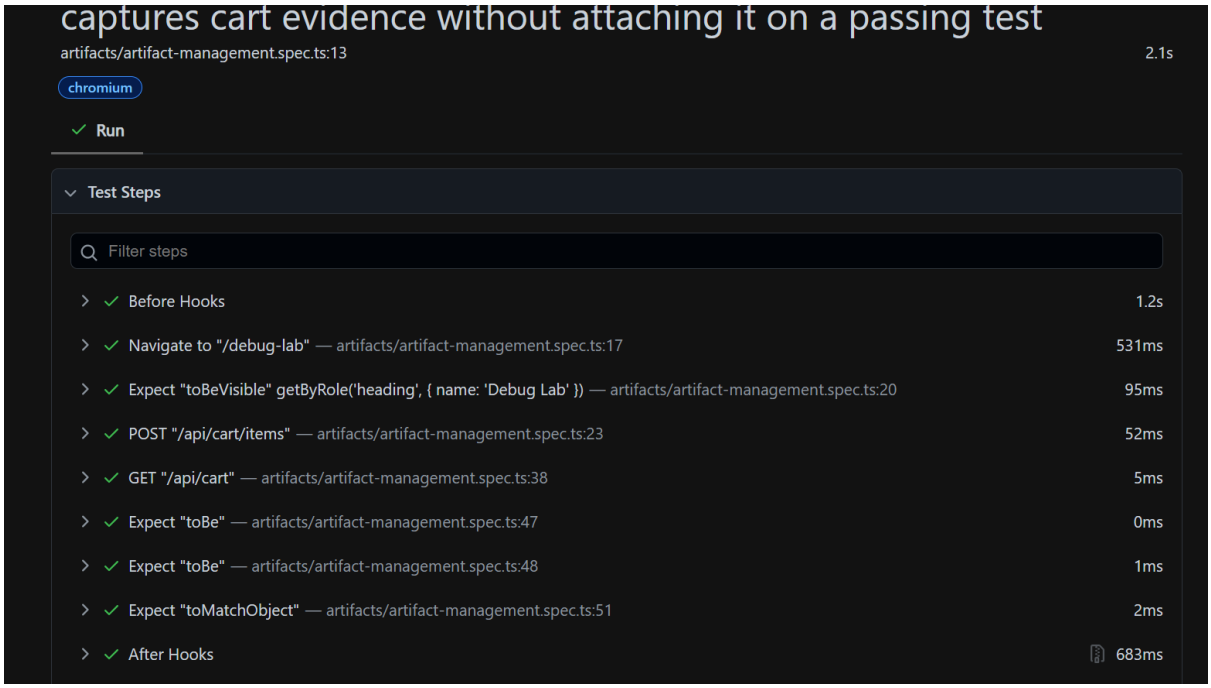
- Playwright HTML Report
- Failure Screenshot
- Cart Evidence

Benefits

Developers can understand failures without reproducing the issue.

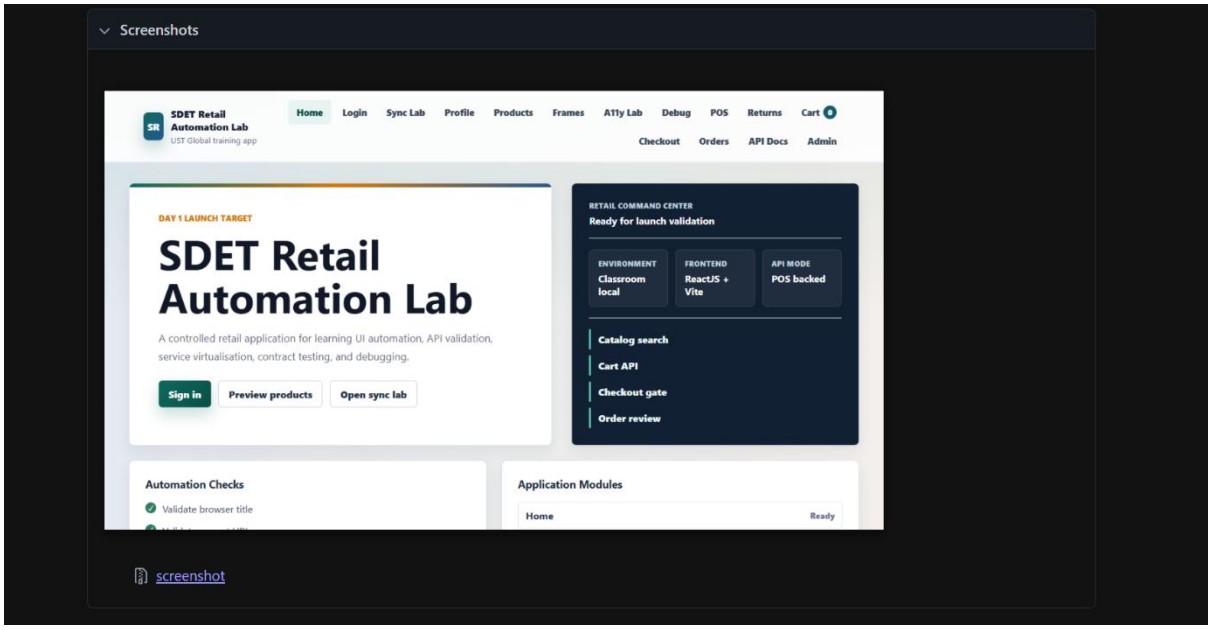
Evidence

- HTML Report (Success)

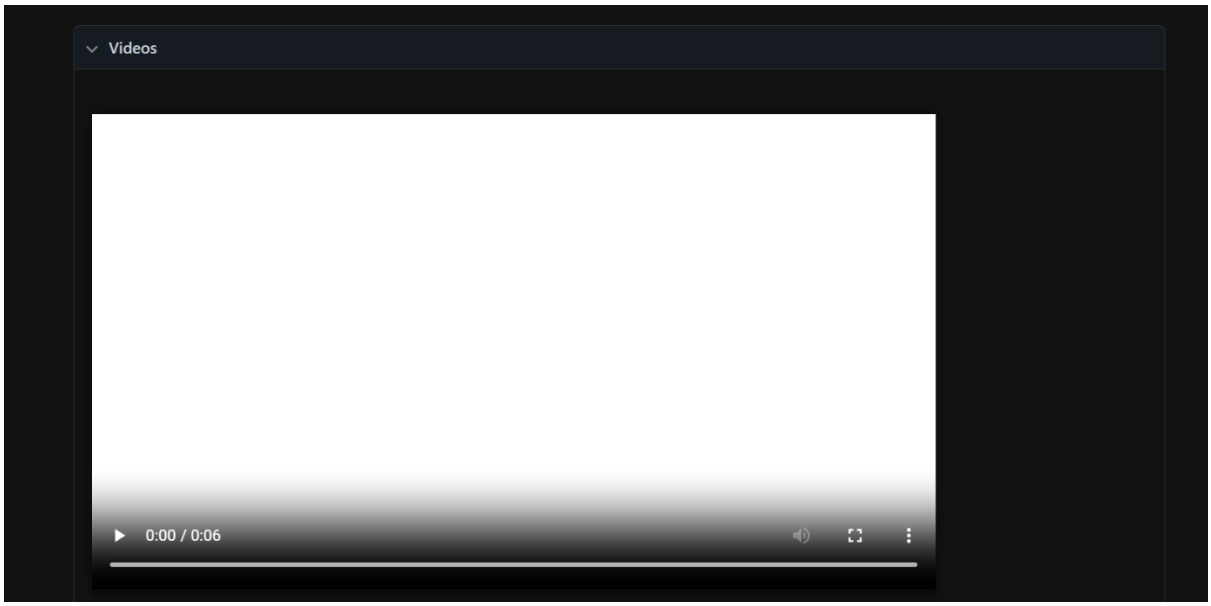


- HTML Report (Failure)

- Failure Screenshot



- Trace Back Screenshot



---

7. Offline & Resilient UI

Command to run in terminal:

npx playwright test network-offline.spec.ts

Test File: network-offline.spec.ts

Scenario 1

Verify offline banner.

Scenario 2

Queue sales while offline.

Scenario 3

Synchronize queued sales after reconnecting.

Scenario 4

Rollback optimistic UI after failed synchronization.

Scenario 5

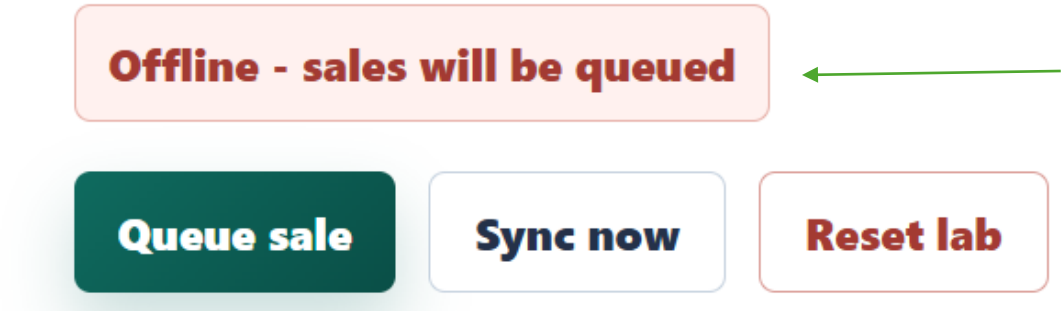
Ensure exactly-once synchronization using Idempotency.

Benefits

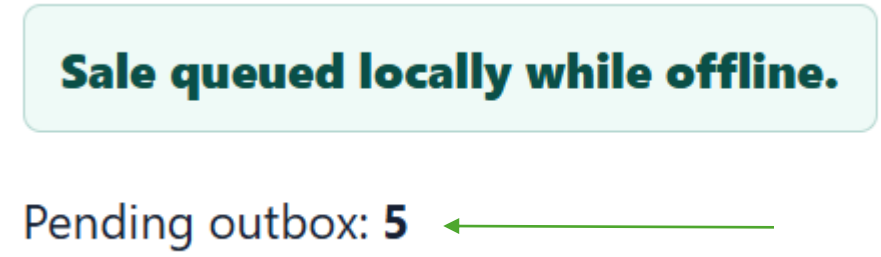
The application continues functioning correctly during network interruptions and safely recovers when connectivity is restored.

Evidence

- Offline banner screenshot



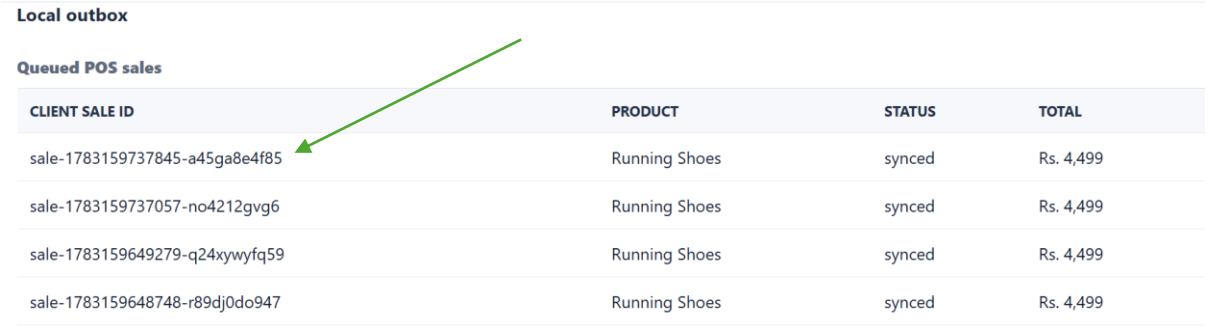
- Queue count screenshot



- Sync completed screenshot

- Rollback screenshot

- Exactly-once synchronization screenshot



---

8. Returns & Refund Testing

Test File: refund-return.spec.ts

The following business scenarios were validated.

Partial Refund

Command to run in terminal:

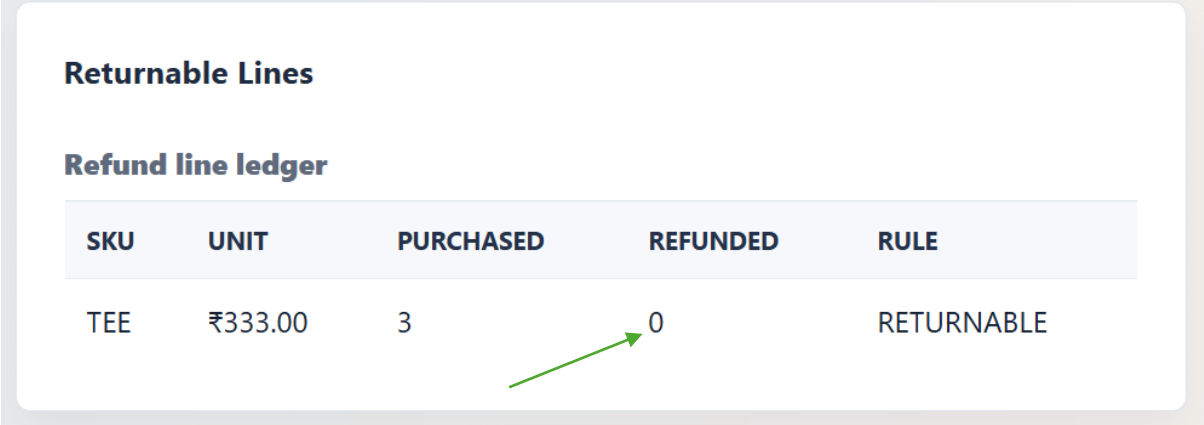
npx playwright test partial-refund.spec.ts

Verified:

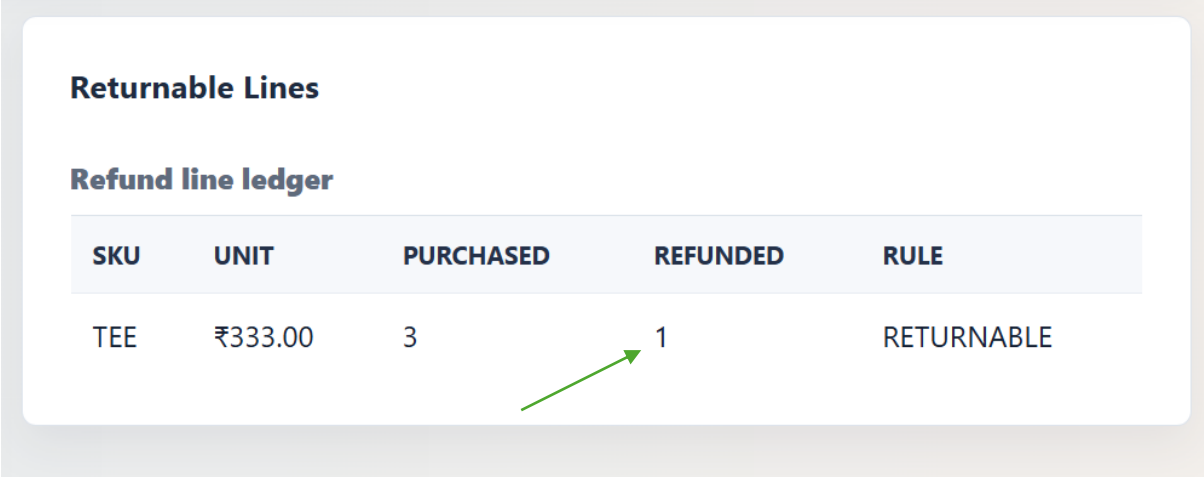
- Refund amount
- Tax calculation
- Ledger update
- UI update

Evidence

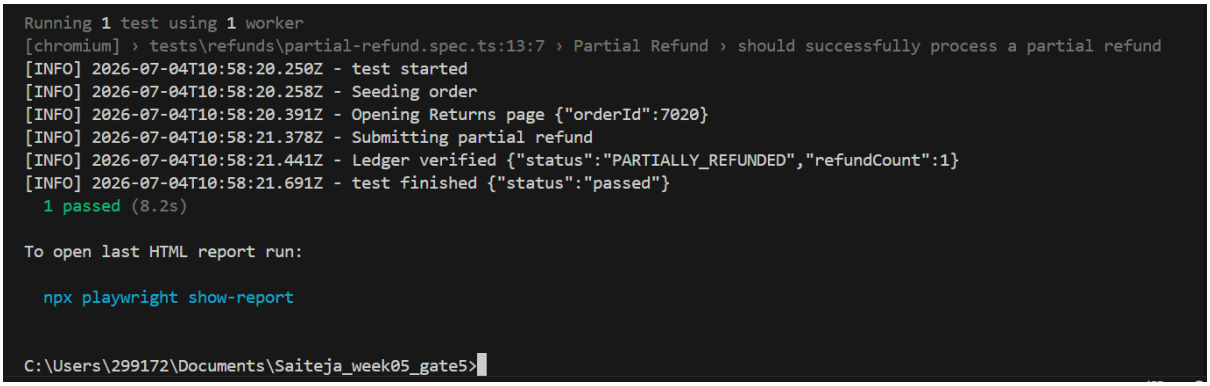
- Before refund screenshot of UI



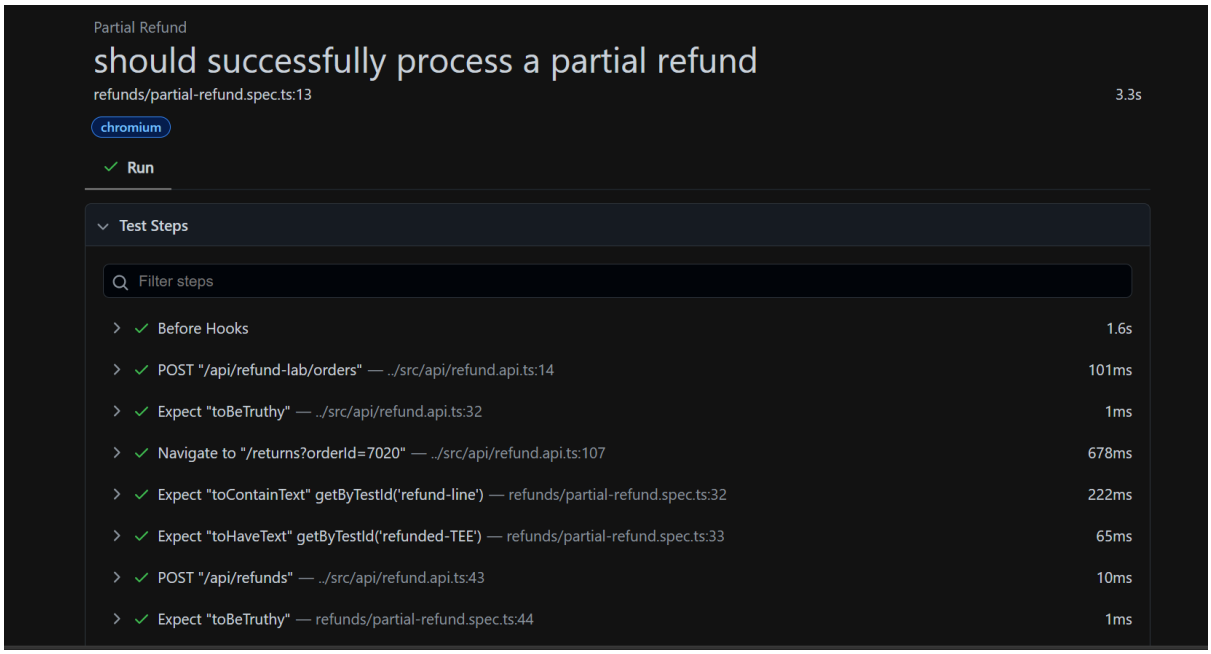
- After refund screenshot of UI



- Terminal Output



- HTML Report



- Evidence

Full Refund

Command to run in terminal:

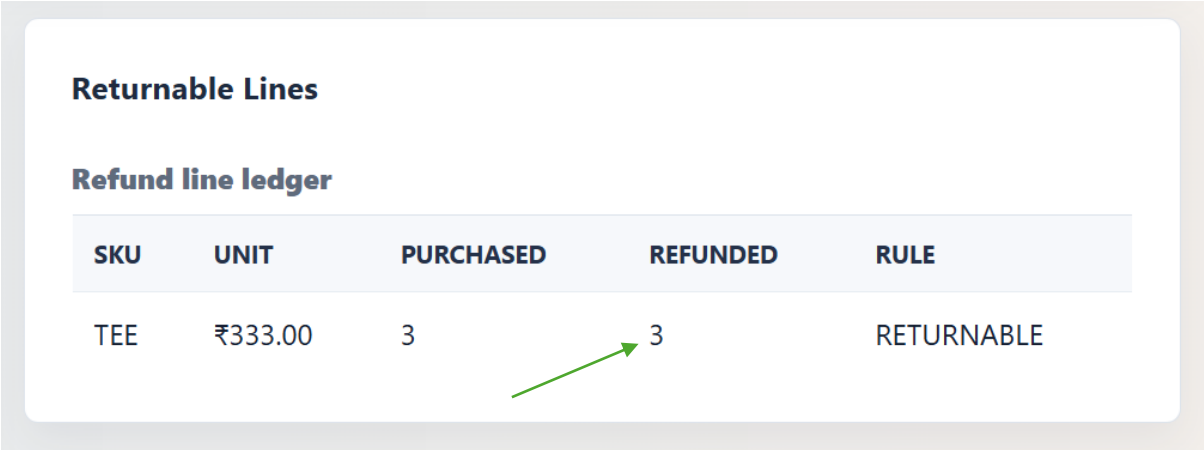
npx playwright test full-refund.spec.ts

Verified:

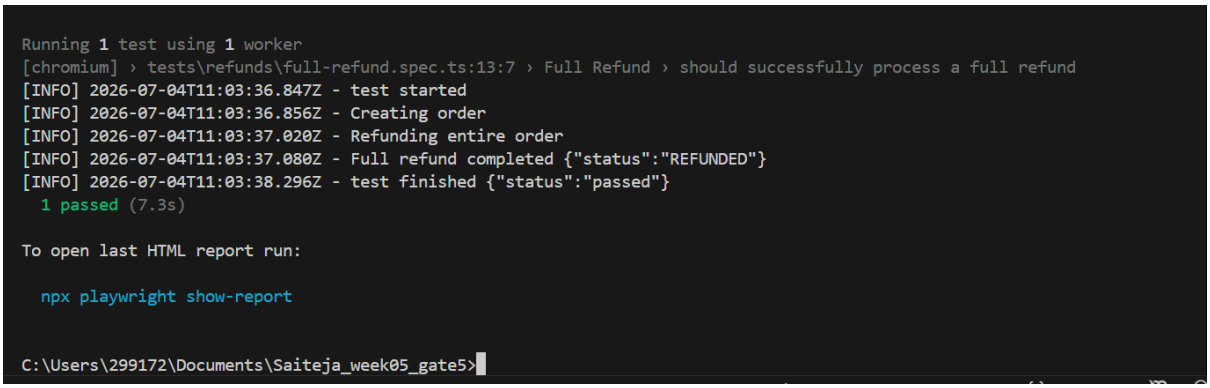
- Complete refund
- Refundable balance becomes zero
- Ledger updated correctly

Evidence

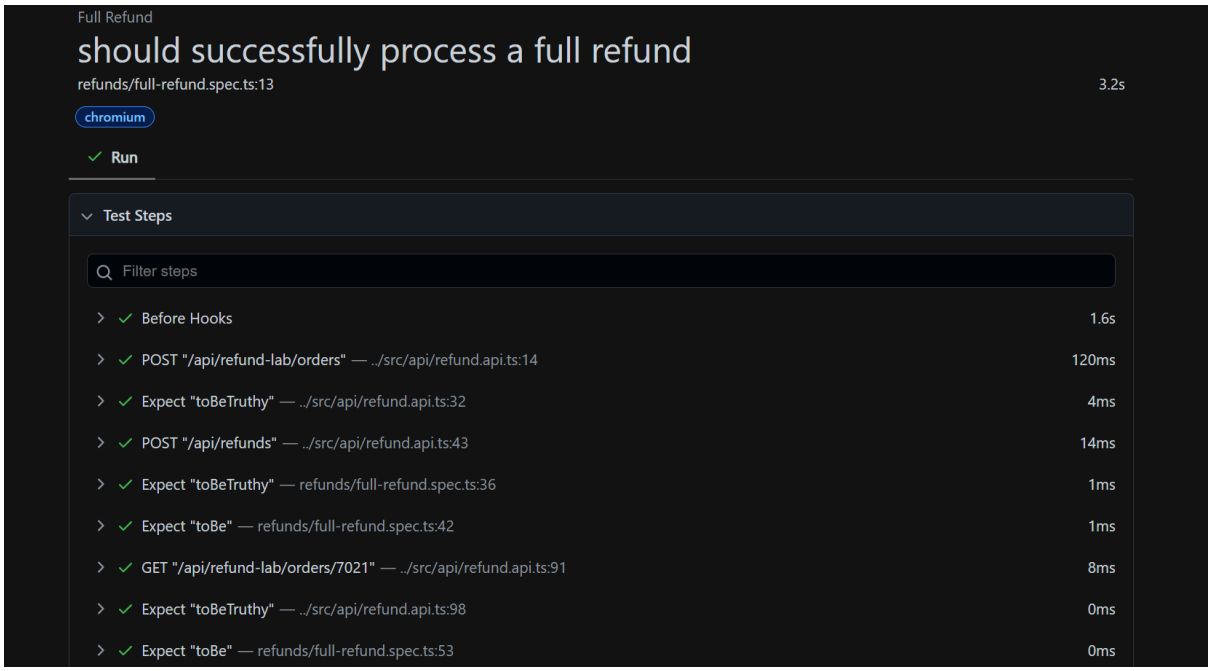
- Full refund screenshot of UI



- Terminal Output



- HTML Report



- Evidence

Refund Eligibility

Command to run in terminal:

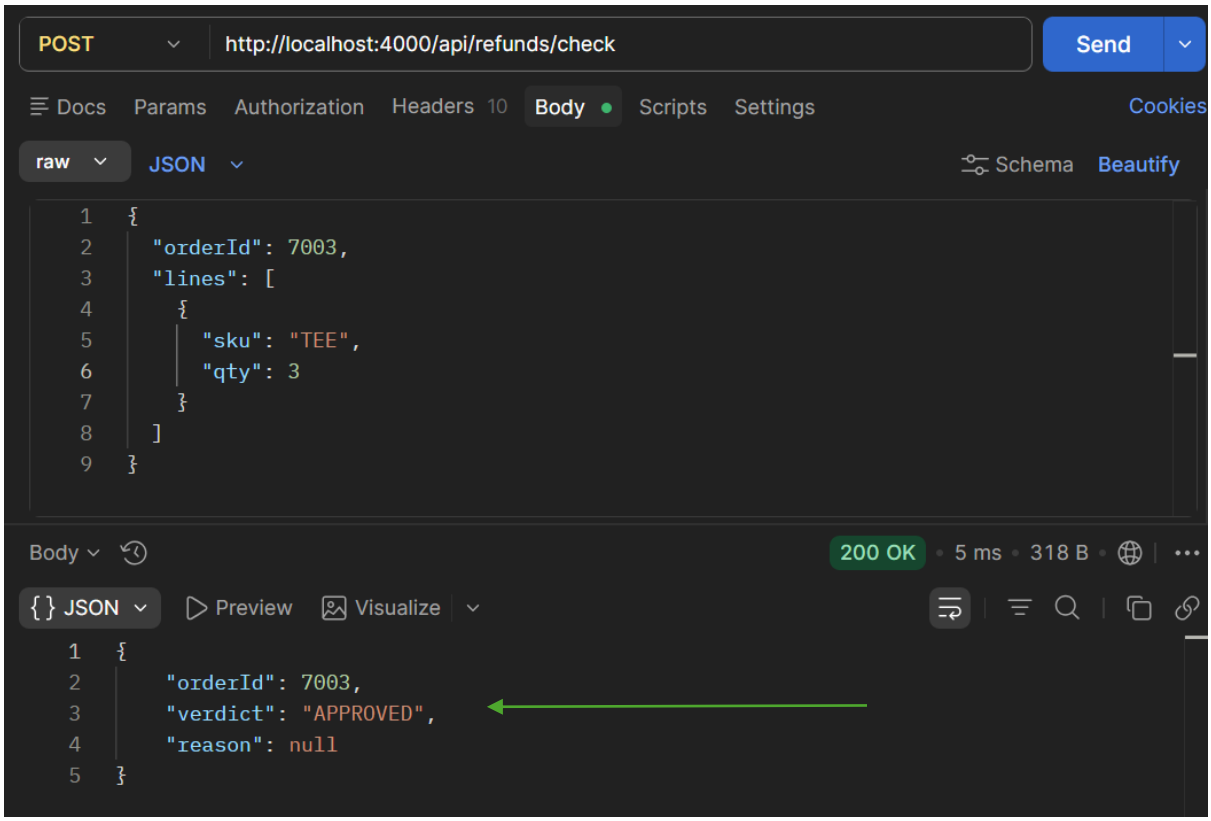
npx playwright test refund-eligibility.spec.ts

Verified:

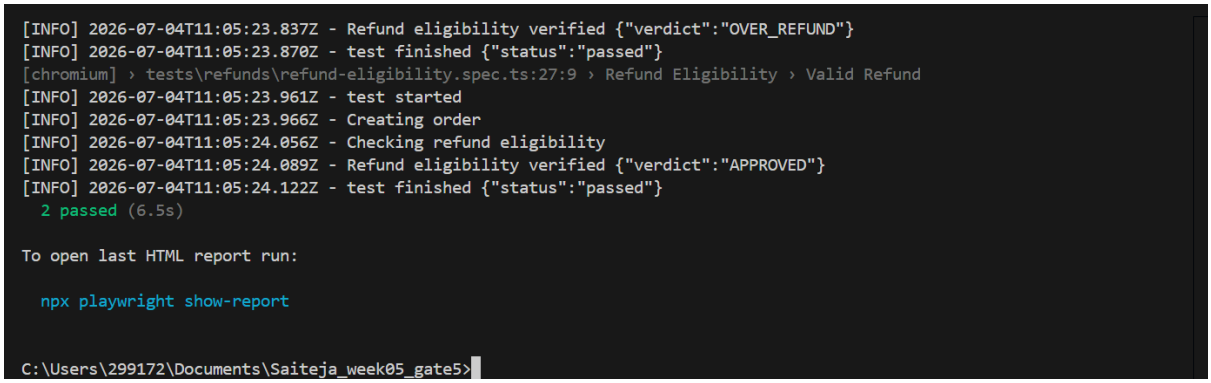
- Valid refund

Evidence

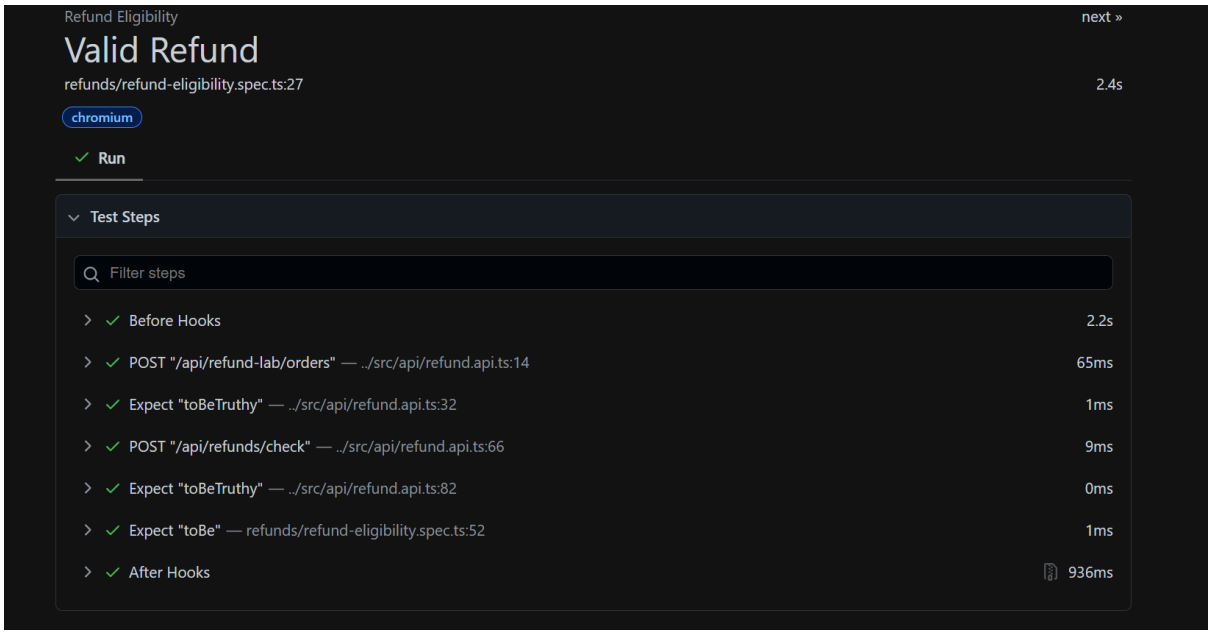
- Refund eligibility screenshot in postman



- Terminal Ouput



- HTML Report



- Evidence

Rejected Refund

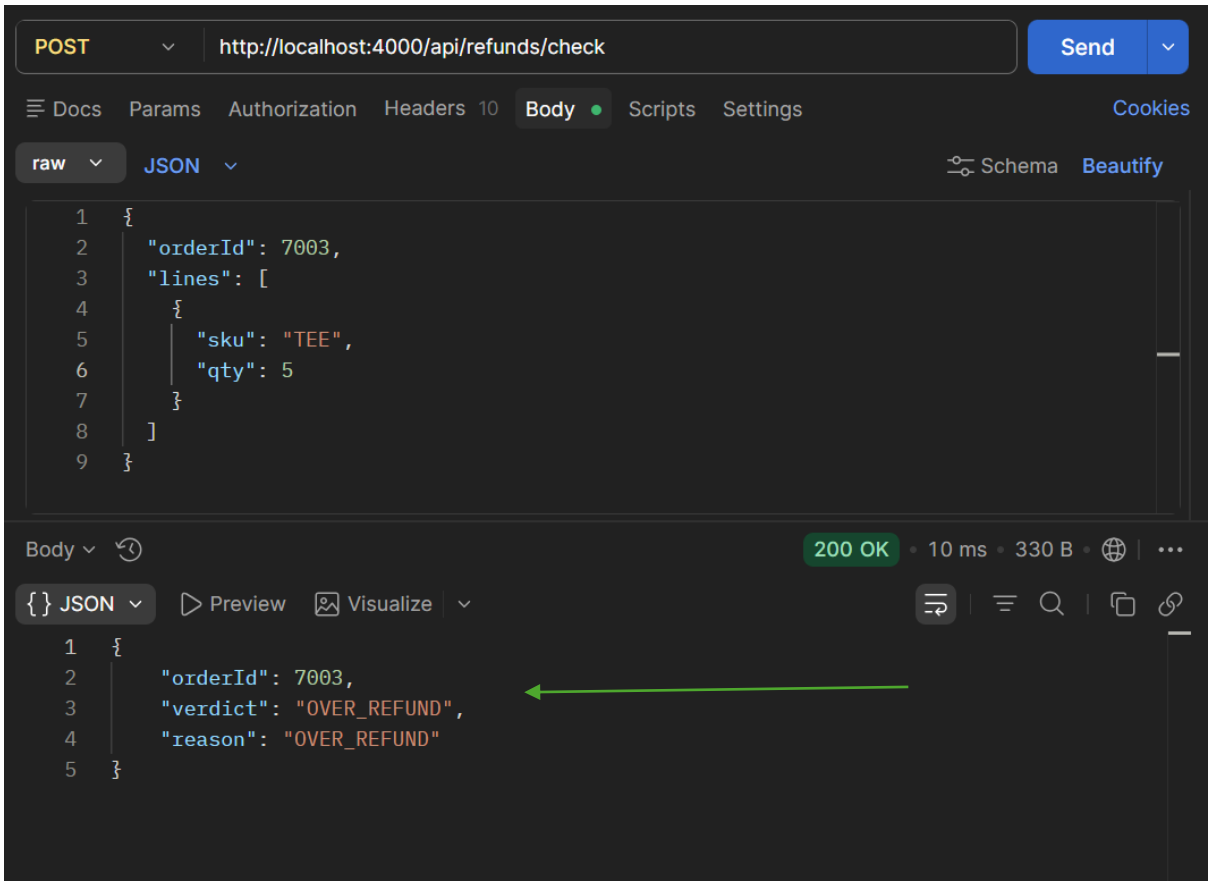
Command to run in terminal:

npx playwright test rejected-refund.spec.ts

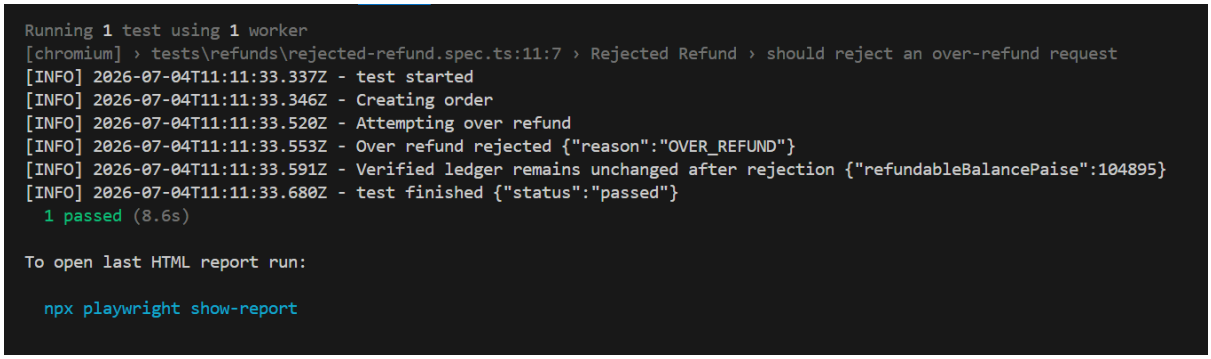
Verified that over-refund requests are rejected.

Evidence

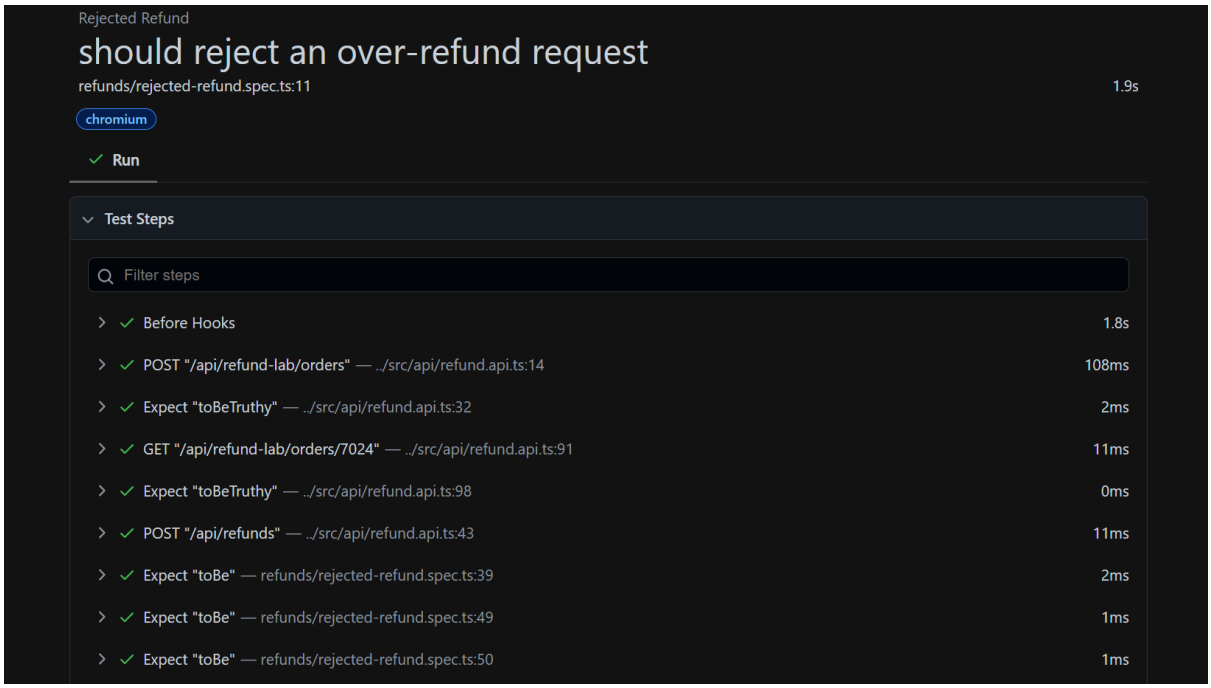
- Rejected refund screenshot from postman



- Terminal Output



- HTML Report



- Evidence

Duplicate Refund

Command to run in terminal:

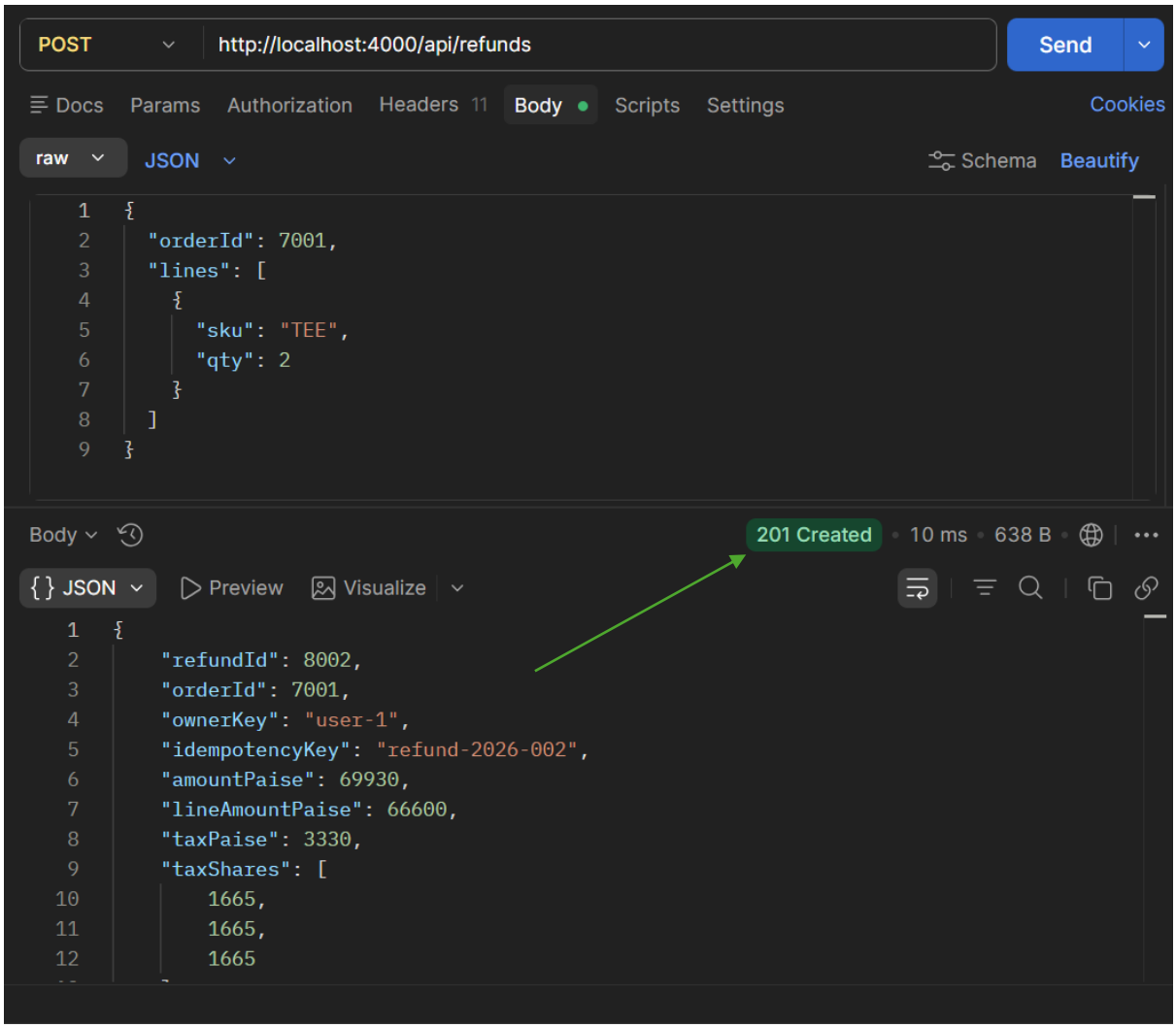
npx playwright test dupicate-refund-idempotency.spec.ts

The same refund request was submitted twice.

The application returned the existing refund instead of creating another one.

Evidence

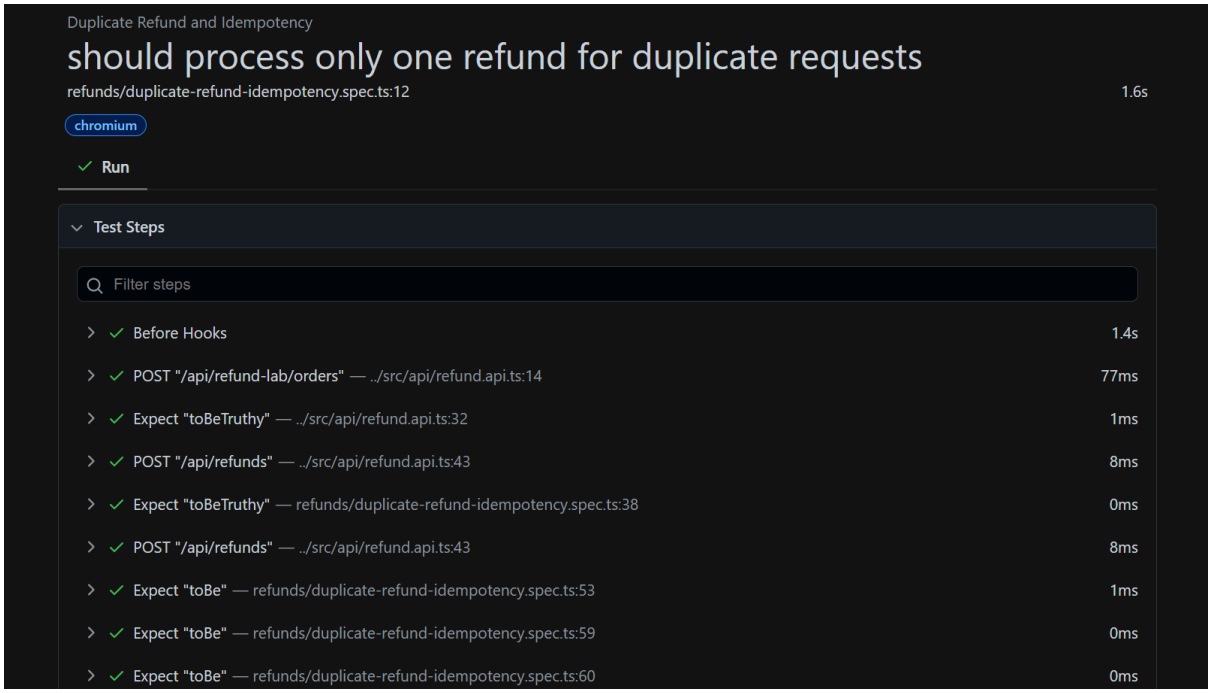
- Duplicate refund
- Screenshot of First time request(Status Code = 201 Created) from Postman



- Terminal Output

```
Running 1 test using 1 worker
…efund-idempotency.spec.ts:12:7 › Duplicate Refund and Idempotency › should process only one refund for duplicate requests
[INFO] 2026-07-04T11:17:13.756Z - test started
[INFO] 2026-07-04T11:17:13.760Z - Creating order
[INFO] 2026-07-04T11:17:13.854Z - Submitting first refund request
[INFO] 2026-07-04T11:17:13.870Z - Submitting duplicate refund request
[INFO] 2026-07-04T11:17:13.901Z - Verified idempotent refund {"refundCount":1,"refundableBalancePaise":69930}
[INFO] 2026-07-04T11:17:13.986Z - test finished {"status":"passed"}
  1 passed (5.6s)

To open last HTML report run:

  npx playwright show-report
```

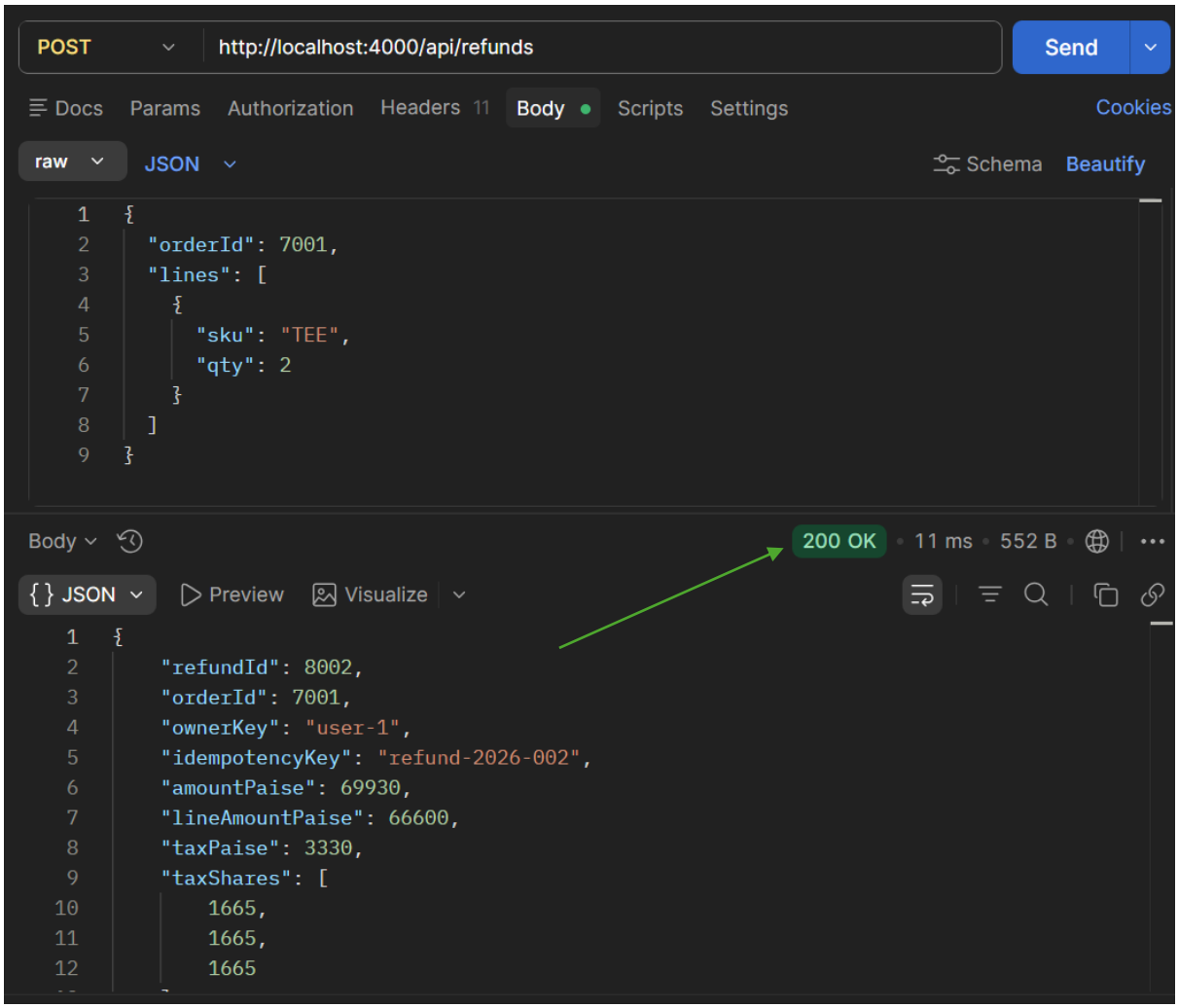
- HTML Report



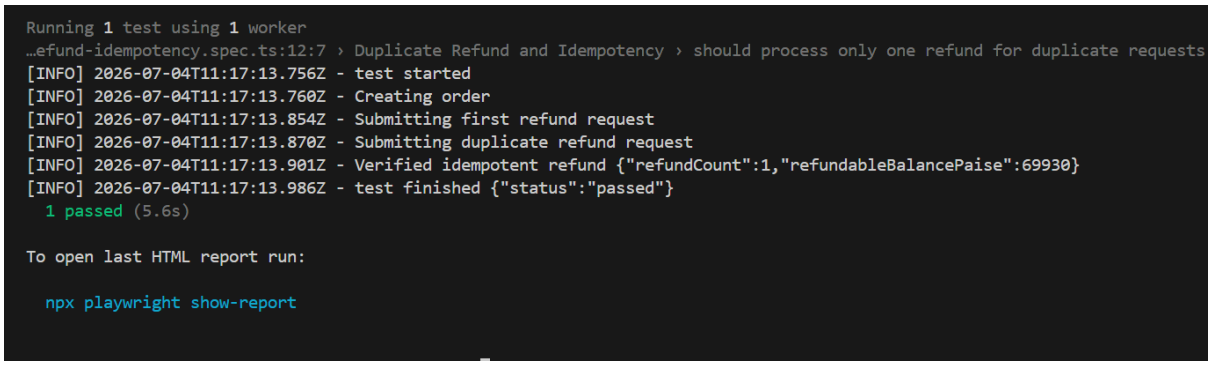
- Evidence


Attachments
order.json
{
"id": 7027,
"orderNumber": "RET-7027",
"ownerKey": "user-1",
"placedOn": "2026-06-29",
"daysAgo": 5,
"returnWindowDays": 30,
"paymentMethod": "Original card",
"lines": [
{
"sku": "TEE",
"name": "Training Tee",
"unitPaise": 33300,
"qty": 3,
"refundedQty": 0,
"finalSale": false,
"nonReturnable": false
}
],
"lineTotalPaise": 99900,
"taxPaise": 4995,
"totalPaise": 104895,
"refundableBalancePaise": 104895,
"refunds": [],
"refundCount": 0,
"lastRefund": null,
"status": "PAID"
}
refund.json
replayRefund.json
ledger.json

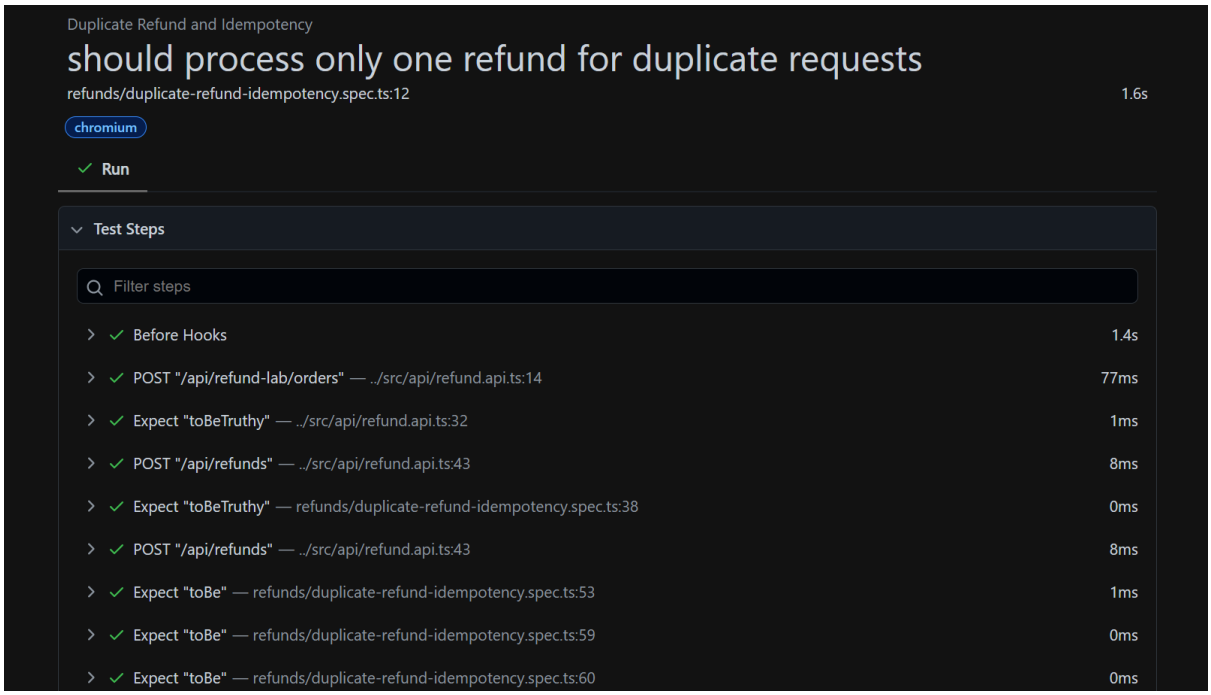
- Screenshot of Second time request(Status Code = 200 OK) from Postman



- Terminal Output



- HTML Report



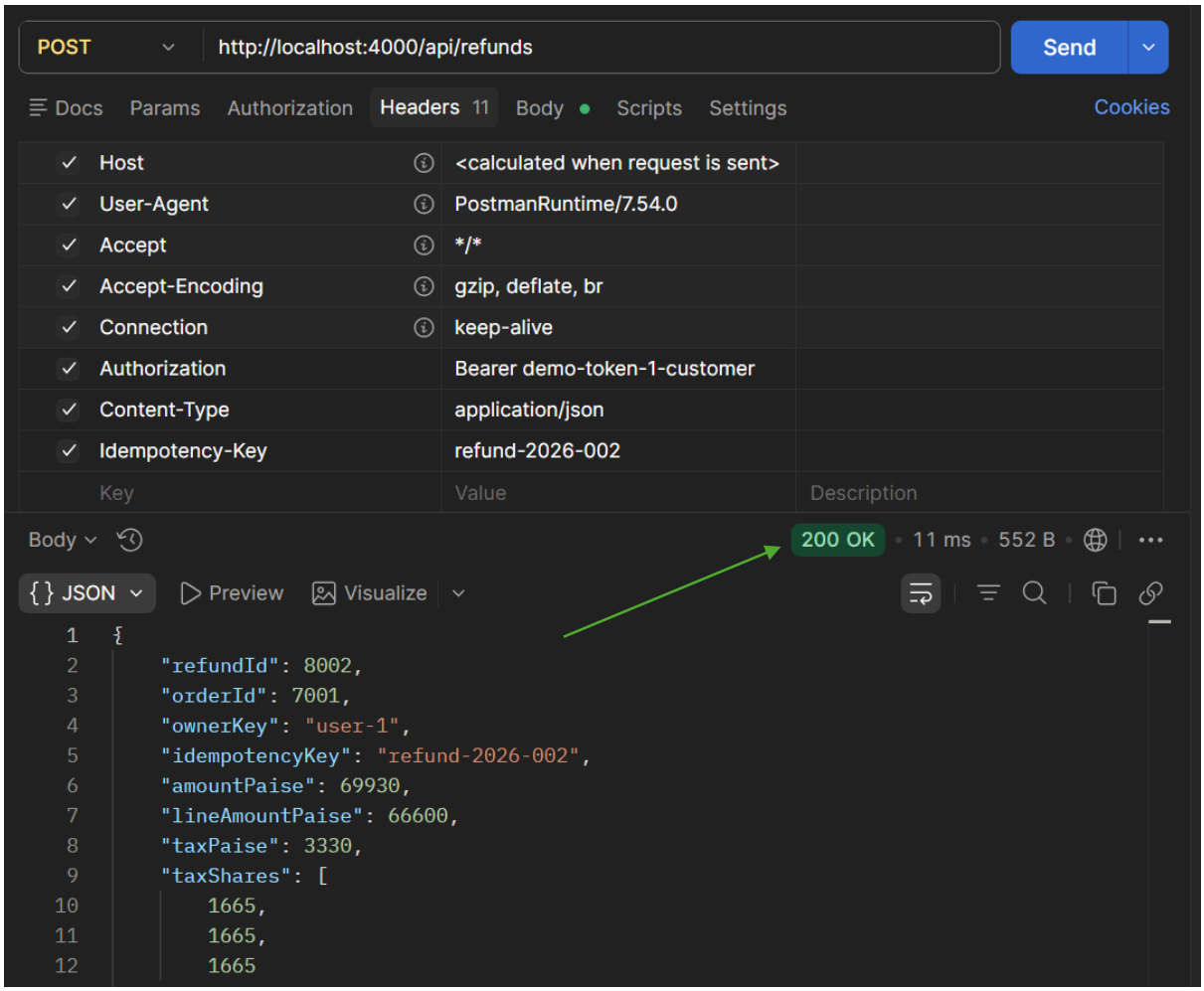
- Evidence

Idempotency

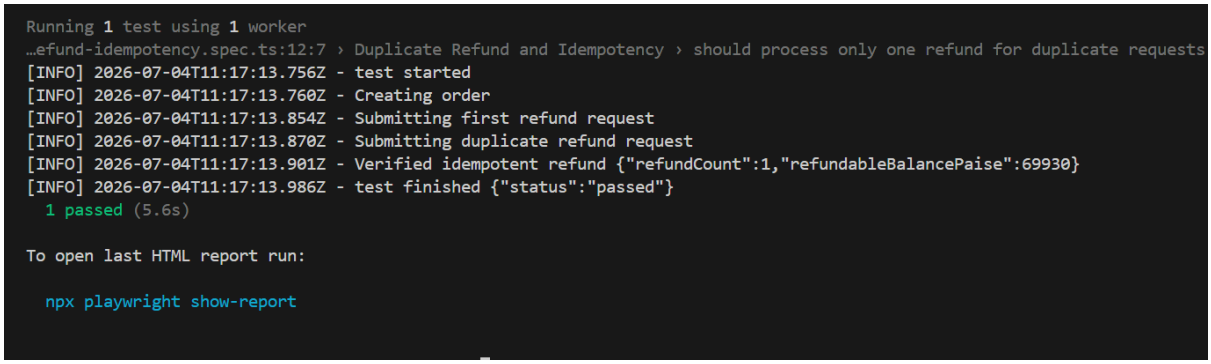
Verified that duplicate requests with the same idempotency key are processed only once.

Evidence

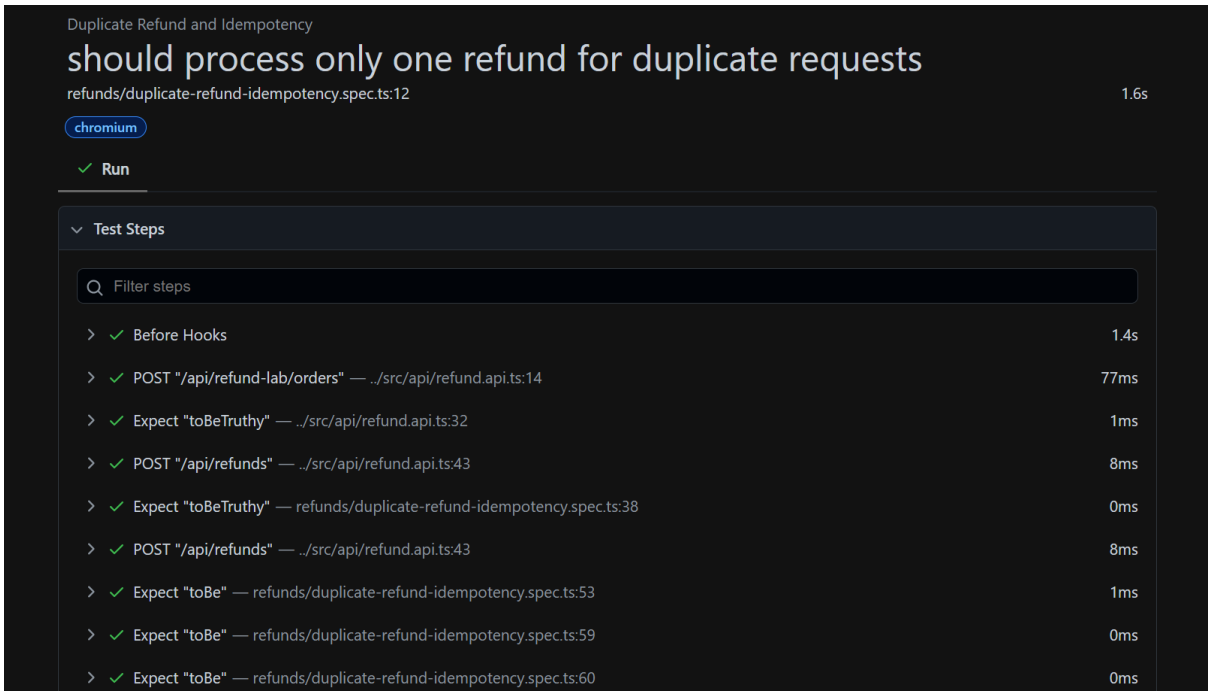
- Idempotency screenshot



- Terminal Output



- HTML Report



- Evidence

Money Precision

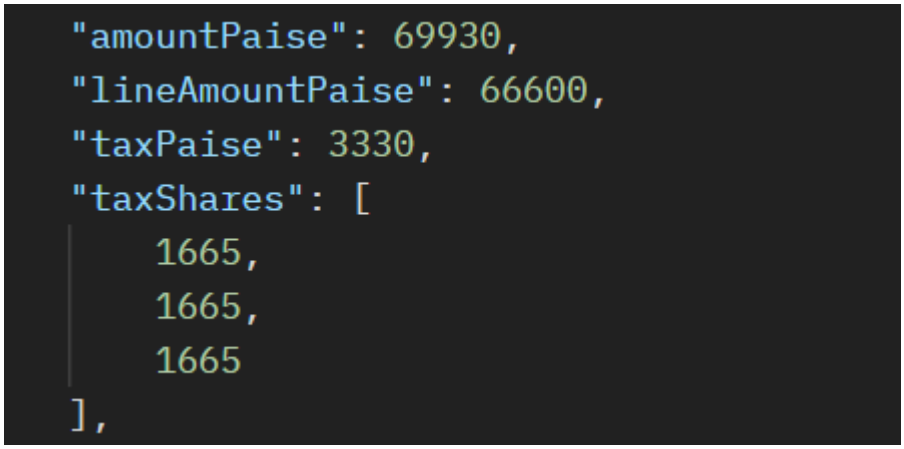
Verified all monetary calculations using **paise** instead of floating-point values.

Validated:

- Refund Amount
- Tax
- Ledger Balance

Evidence

- Money precision screenshot



---

9. Failure Analysis

Every issue in this assessment follows the same debugging approach.

Symptom

Identify the observed failure.

Diagnosis

Analyze logs, assertions and application behavior to determine the root cause.

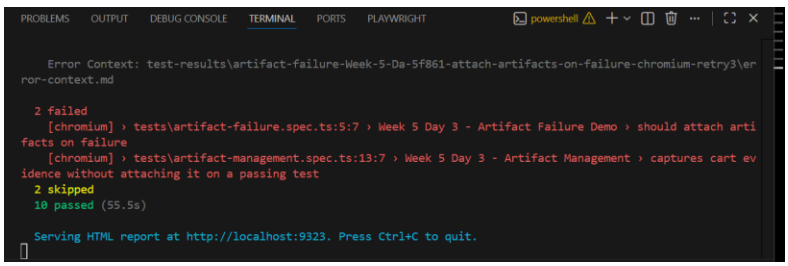
Fix

Apply the appropriate solution instead of temporary workarounds.

Evidence

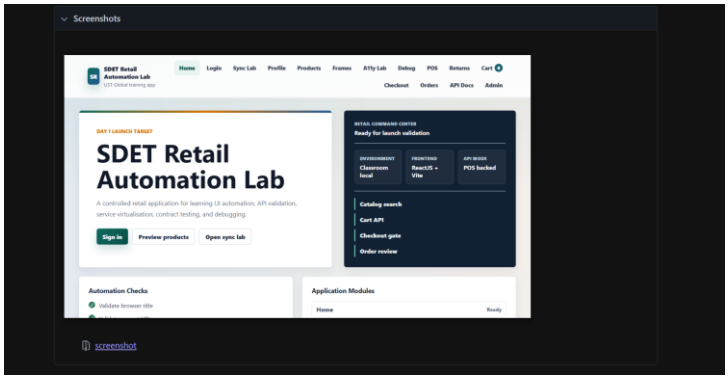
Validate the fix using:

- Screenshot

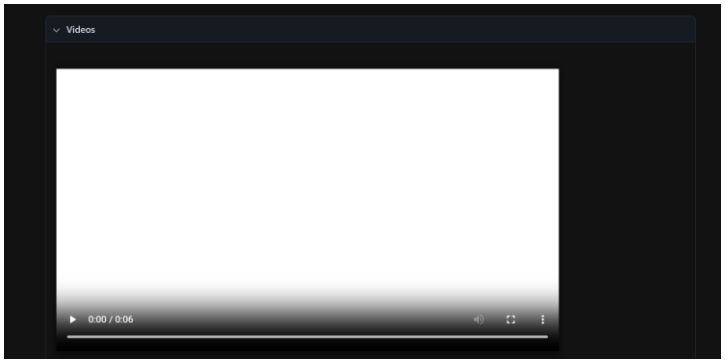


- HTML Report

- Failure Screenshot



- Trace Back Video



---

10. Debugging Maturity

This assessment demonstrates debugging maturity through:

- Identifying flaky tests
- Diagnosing root causes
- Structured diagnostic logging
- Artifact collection

- Offline resilience testing
- Business rule validation
- Refund validation
- Idempotency testing
- Financial validation using paise

---

11. Challenges Faced

- Diagnosing flaky tests
- Handling offline scenarios
- Validating idempotent requests
- Preventing duplicate refunds
- Testing financial calculations
- Capturing useful debugging evidence

---

12. Conclusion

This assessment successfully demonstrates stable Playwright automation by focusing on diagnosing issues, implementing reliable solutions, collecting meaningful evidence, and clearly explaining the results. The project validates critical business scenarios such as offline operations, refund processing, artifact management, and structured diagnostic logging while improving overall debugging maturity.

---

13. Evidence Summary

Assessment Area	**Evidence to Include**
Flakiness Analysis	Before failure screenshot, After fix screenshot, HTML Report
Diagnostic Logging	Correlation ID logs, Structured logs, Redacted logs, Trace flow screenshot

Assessment Area	**Evidence to Include**
Artifact Management	HTML Report (Pass), HTML Report (Fail), Failure Screenshot, Trace Back Screenshot
Offline & Resilient UI	Offline banner, Queue count, Rollback, Sync success
Returns & Refund Testing	Partial refund, Full refund, Rejected refund, Duplicate refund, Idempotency, Money precision